

Day 10: Bayes' Theorem Python Automation Notes

MANUAL ITERATION VS. BUILT-IN VECTORIZATION LAB BLUEPRINT

1. Introduction to Python Programming for Bayes' Theorem

To implement Bayes' Theorem inside enterprise analytical environments, manual calculation is replaced by programmatic automation using the Python Pandas library. This blueprint covers two distinct engineering pathways: a Detailed Explanatory Loop and a High-Performance Built-in Vectorization script.

Bhai, Bayes' Theorem ko enterprise analytical scale par automation se chalane ke liye hum manual calculations ko Python Pandas library se replace karte hain. Is blueprint mein hum dono tarike dekhenge —ek poora samajhne wala Detailed Explanatory Loop aur dusra superfast High-Performance Built-in Vectorization script.

2. Approach 1: Detailed Explanatory & Step-by-Step Loop Code

This method is structured to map internal calculations line-by-line using horizontal positional alignment. It isolates targeted numerators and iterates through rows using the explicit `.iterrows()` framework to mirror traditional mathematical verification.

Yeh method ek-ek step ko line-by-line horizontal alignment ke sath dimaag mein set karta hai. Yeh target numerators ko alag karta hai aur `.iterrows()` framework ka use karke table ke har ek row par manually calculation map karta hai taaki maths poori tarah verify ho sake.

```

import pandas as pd

# STEP 1: ECOSYSTEM DATA SET DESIGN (Prior Shares & Core Rates)
data = {
    "Channel": ["Premium Members (A1)", "Guest Checkouts (A2)", "New VPN Accounts (A3)"],
    "Prior_Probability_P_A": [0.60, 0.30, 0.10],
    "Internal_Fraud_Rate_P_Fraud_Given_A": [0.005, 0.03, 0.15],
    "Detection_Accuracy_P_Alert_Given_Fraud": [0.80, 0.90, 0.98]
}

df = pd.DataFrame(data)

# STEP 2: MULTI-FACTOR SEQUENCE EXECUTION (The Core Multiplication Rule)
df["Real_Alert_Volume_Numerator"] = (
    df["Prior_Probability_P_A"] * df["Internal_Fraud_Rate_P_Fraud_Given_A"] *
    df["Detection_Accuracy_P_Alert_Given_Fraud"]
)

# STEP 3: AGGREGATING TOTAL SAMPLE SPACE (The Combined Denominator Pool)
total_alerts_denominator = df["Real_Alert_Volume_Numerator"].sum()

# STEP 4: MAPPING POSTERIOR PROBABILITIES (Division & Scaling)
df["Posterior_Probability_P_A_Given_Alert"] = df["Real_Alert_Volume_Numerator"] /
total_alerts_denominator
df["Posterior_Percentage"] = df["Posterior_Probability_P_A_Given_Alert"] * 100

# STEP 5: DISPLAYING LOGICAL OUTPUT & METRICS AUDIT VIA ITERROWS
print("--- FINAL POSTERIOR REPORT ENGINE (BAYES' THEOREM OUTPUT) ---")
for index, row in df.iterrows():
    print(f"\nTarget: {row['Channel']}")
    print(f" -> Isolate Numerator (Absolute Real Volume) = {row['Real_Alert_Volume_Numerator']:.4f}")
    print(f" -> Total Denominator Pool (All Alert Sources) = {total_alerts_denominator:.4f}")
    print(f" -> Calculation: {row['Real_Alert_Volume_Numerator']:.4f} / {total_alerts_denominator:.4f}")
    print(f" -> Final Posterior Risk Percentage = {row['Posterior_Percentage']:.2f}%")

print("\n" + "="*50)
highest_risk = df.loc[df["Posterior_Percentage"].idxmax()]
print(f"the primary auditing target should immediately be: {highest_risk['Channel']}")
print(f"holding an absolute risk weight of {highest_risk['Posterior_Percentage']:.2f}%")

```

```
%.")  
print("="*50)
```

3. Approach 2: Compact Built-in Vectorization Code (High Performance Shortcut)

This operational layout uses pure Pandas vectorization shortcuts. By executing column-level division arrays and using the built-in `.idxmax()` pointer, it eradicates row loops completely, running directly via the fast C-engine backend of Pandas.

Yeh operational layout pure Pandas vectorization ka shortcut use karta hai. Column-level division arrays aur built-in `.idxmax()` pointer ka use karke yeh row loops ko poori tarah khatam kar deta hai, jisse code seedhe Pandas ke superfast C-engine backend par execute hota hai.

```
import pandas as pd

# 1. Base Dataset (Raw Dictionary input to DataFrame)
data = {
    "Channel": ["Premium Members (A1)", "Guest Checkouts (A2)", "New VPN Accounts (A3)"],
    "P_A": [0.60, 0.30, 0.10],
    "P_Fraud_A": [0.005, 0.03, 0.15],
    "P_Alert_Fraud": [0.80, 0.90, 0.98]
}
df = pd.DataFrame(data)

# THE SHORTCUT BUILT-IN ENGINE (ONLY 4 LINES CORES)
df["Numerator"] = df["P_A"] * df["P_Fraud_A"] * df["P_Alert_Fraud"]
df["Risk_%"] = (df["Numerator"] / df["Numerator"].sum()) * 100
highest_risk = df.loc[df["Risk_%"].idxmax()]

# DISPLAY THE COMPACT REPORT
print("--- QUICK BUSINESS REPORT ---")
print(df[["Channel", "Risk_%"]].to_string(index=False, float_format="%.2f%"))
print("="*50)
print(f"IMMEDIATE TARGET: {highest_risk['Channel']} ({highest_risk['Risk_%']:.2f}%)"
print("="*50)
```

4. Core Code Component Breakdown & Mental Mapping

The Positional Alignment Rule: The `pd.DataFrame(data)` object maps vertical lists horizontally based on position index. Column keys like `"Channel"` bind structural identity to row values across every mathematical operation.

Positional Alignment ka Niyam: `pd.DataFrame(data)` object sabhi vertical lists ko unke structural position index ke mutabik horizontally ek sath lock karta hai. Is wajah se "Channel" column poore mathematical calculations mein apni sateek pehchan banaye rakhta hai.

Key Built-in Functions Architecture Summary:

- `.sum()`: Directly processes matrix arrays to calculate totals without manual addition loops.
- `.idxmax()`: Scans numerical arrays instantly and points out the exact row location holding the maximum value.
- `.loc[...]`: Directly locks onto the specific horizontal row index coordinates to extract comprehensive dataset info.